Aufgabe 1: Programmierbare Logikbausteine (PLA)

Programmierbare Logikbausteine (PLA) – Formelsammlung & Übersicht

1. Struktur eines PLA (Programmable Logic Array):
Ein PLA realisiert boolesche Schaltnetze zweistufig in disjunktiver Normalform (DNF):

- **Eingänge:** Stehen in wahrer Form ($\mathrm{x_i}$) und invertierter Form ($\neg \mathrm{x_i}$) zur Verfügung.
- **Konjunktionsmatrix (AND):** Programmierbar. Bildet Produktterme $\mathrm{P_k}$ durch wählbare Kreuzungspunkte.
- **Disjunktionsmatrix (OR):** Programmierbar. Verknüpft Produktterme zu Ausgängen $\mathrm{y_j} = \bigvee \mathrm{P_k}$.

2. Berkeley-Espresso-Format (.pla):
`.i <Eingänge> .o <Ausgänge> .p <Produktterme> .e`
In den Produktzeilen: `0` = invertiert ($\neg \mathrm{x}$), `1` = nicht-invertiert (x), `-` = Don't Care (beliebig), gefolgt von der Ausgangsmaske (`1` = aktiv, `0` = inaktiv).

3. Bausteinvergleich:
PROM: AND fest (2^{n}-Decodierer), OR programmierbar | **PAL:** AND programmierbar, OR fest | **PLA:** AND und OR programmierbar (erlaubt Term-Sharing).

Simulator ↗

1.) Espresso-Format zu DNF & Wahrheitstabelle

Gegeben sei folgende PLA-Spezifikation im Berkeley-Espresso-Format mit 4 Eingängen ($\mathrm{x_1}, \mathrm{x_2}, \mathrm{x_3}, \mathrm{x_4}$) und 1 Ausgang ($\mathrm{y}$):

```
.i 4
.o 1
.ilb x1 x2 x3 x4
.ob y
.p 3
000- 1
01-1 1
1--0 1
.e
```

Aufgaben:

1. Geben Sie die boolesche Funktionsgleichung y in disjunktiver Normalform (DNF) an.
2. Bestimmen Sie, für welche Belegungen von $(\mathrm{x_1}, \mathrm{x_2}, \mathrm{x_3}, \mathrm{x_4})$ der Ausgang $\mathrm{y} = 1$ wird.

Simulator ↗

2.) Matrix-Programmierung (AND- & OR-Matrix)

Gegeben seien zwei Funktionen y_1, y_2 mit 3 Eingängen (x_1, x_2, x_3):

$$y_1 = (\neg x_1 \wedge x_2) \vee (x_1 \wedge x_3)$$

$$y_2 = (\neg x_1 \wedge x_2) \vee (\neg x_2 \wedge \neg x_3)$$

Aufgaben:

1. Wie viele Zeilen in der AND-Matrix (Produktterme) werden minimal benötigt?
2. Geben Sie für jeden Produktterm an, welche Eingangsleitungen (x_i bzw. $\neg x_i$) in der AND-Matrix verbunden werden.
3. Beschreiben Sie die Verschaltung der OR-Matrix zur Erzeugung von y_1 und y_2.

Simulator ↗

3.) Produktterm-Sharing zur Hardware-Minimierung

In programmierbaren Logikbausteinen (PLAs) ist die Anzahl der zur Verfügung stehenden Zeilen in der Konjunktionsmatrix (Produktterm-Budget) fest vorgegeben (z. B. maximal 4 oder 8 Produktterme).
Frage: Erklären Sie, warum bei der Minimierung für PLAs eine **gemeinsame Minimierung mehrerer Ausgänge** (Multi-Output Minimization) oft zu einem hardware-effizienteren Ergebnis führt als die getrennte KV-Minimierung jedes einzelnen Ausgangs.

Simulator ↗

4.) Don't-Cares in der Eingabe (BCD-Pseudotetraden)

Ein 4-Bit BCD-Signal ($\mathrm{x_3x_2x_1x_0}$, mit $\mathrm{x_3}$ als MSB) codiert Dezimalziffern von 0 bis 9 ($0000_2 \ldots 1001_2$).
Ein Fehlerdetektor ERR soll eine 1 ausgeben, wenn eine unzulässige Pseudotetrade ($10 \ldots 15$) anliegt ($1010_2 \ldots 1111_2$).
Aufgaben:

1. Formulieren Sie die minimierte DNF für ERR unter Ausnutzung von Don't Cares.
2. Geben Sie die entsprechende Spezifikation im Berkeley-Espresso-Format an. Wie viele Produktterme werden im PLA benötigt?

Simulator ↗

5.) Systemvergleich: PROM vs. PAL vs. PLA

In der Digitaltechnik stehen verschiedene Architekturen programmierbarer Bausteine zur Verfügung.

Aufgaben:

1. Beschreiben Sie die Unterschiede bezüglich der Programmierbarkeit von AND- und OR-Matrix bei PROM, PAL und PLA.
2. Warum hat sich in der industriellen Praxis das PAL-Konzept historisch stärker verbreitet als das flexiblere PLA?

Programmierbare Logikbausteine (PLA) – Formelsammlung & Übersicht

1. Struktur eines PLA (Programmable Logic Array):
Ein PLA realisiert boolesche Schaltnetze zweistufig in disjunktiver Normalform (DNF):

- **Eingänge:** Stehen in wahrer Form ($\mathrm{x_i}$) und invertierter Form ($\neg \mathrm{x_i}$) zur Verfügung.
- **Konjunktionsmatrix (AND):** Programmierbar. Bildet Produktterme $\mathrm{P_k}$ durch wählbare Kreuzungspunkte.
- **Disjunktionsmatrix (OR):** Programmierbar. Verknüpft Produktterme zu Ausgängen $\mathrm{y_j} = \bigvee \mathrm{P_k}$.

2. Berkeley-Espresso-Format (.pla):
`.i <Eingänge> .o <Ausgänge> .p <Produktterme> .e`
In den Produktzeilen: `0` = invertiert ($\neg \mathrm{x}$), `1` = nicht-invertiert (x), `-` = Don't Care (beliebig), gefolgt von der Ausgangsmaske (`1` = aktiv, `0` = inaktiv).

3. Bausteinvergleich:
PROM: AND fest (2^{n}-Decodierer), OR programmierbar | **PAL:** AND programmierbar, OR fest | **PLA:** AND und OR programmierbar (erlaubt Term-Sharing).

Simulator ↗

1.) Espresso-Format zu DNF & Wahrheitstabelle

Gegeben sei folgende PLA-Spezifikation im Berkeley-Espresso-Format mit 4 Eingängen ($\mathrm{x_1}, \mathrm{x_2}, \mathrm{x_3}, \mathrm{x_4}$) und 1 Ausgang ($\mathrm{y}$):

```
.i 4
.o 1
.ilb x1 x2 x3 x4
.ob y
.p 3
000- 1
01-1 1
1--0 1
.e
```

Aufgaben:

1. Geben Sie die boolesche Funktionsgleichung y in disjunktiver Normalform (DNF) an.
2. Bestimmen Sie, für welche Belegungen von $(\mathrm{x_1}, \mathrm{x_2}, \mathrm{x_3}, \mathrm{x_4})$ der Ausgang $\mathrm{y} = 1$ wird.

Musterlösung Szenario 1:

1. **DNF-Funktionsgleichung:** Jede Zeile mit Ausgang `1` entspricht einem Konjunktionsterm (Produktterm):

- Zeile 1: `000-` $\implies \mathrm{P}_1 = \neg \mathrm{x}_1 \wedge \neg \mathrm{x}_2 \wedge \neg \mathrm{x}_3$
- Zeile 2: `01-1` $\implies \mathrm{P}_2 = \neg \mathrm{x}_1 \wedge \mathrm{x}_2 \wedge \mathrm{x}_4$
- Zeile 3: `1--0` $\implies \mathrm{P}_3 = \mathrm{x}_1 \wedge \neg \mathrm{x}_4$

Gesamtfunktion: $\mathrm{y} = \mathrm{P}_1 \vee \mathrm{P}_2 \vee \mathrm{P}_3 = (\neg \mathrm{x}_1 \wedge \neg \mathrm{x}_2 \wedge \neg \mathrm{x}_3) \vee (\neg \mathrm{x}_1 \wedge \mathrm{x}_2 \wedge \mathrm{x}_4) \vee (\mathrm{x}_1 \wedge \neg \mathrm{x}_4)$.

2. **Eingangsbelegungen mit $\mathrm{y} = 1$:**

- P_1 ist 1 für: $(0,0,0,0)$ und $(0,0,0,1)$ (Minterme 0, 1)
- P_2 ist 1 für: $(0,1,0,1)$ und $(0,1,1,1)$ (Minterme 5, 7)
- P_3 ist 1 für: $(1,0,0,0)$, $(1,0,1,0)$, $(1,1,0,0)$, $(1,1,1,0)$ (Minterme 8, 10, 12, 14)

$\mathrm{y} = 1$ gilt genau für die 8 Minterme: $\{0, 1, 5, 7, 8, 10, 12, 14\}$.

Simulator ↗

2.) Matrix-Programmierung (AND- & OR-Matrix)

Gegeben seien zwei Funktionen $\mathrm{y}_1, \mathrm{y}_2$ mit 3 Eingängen $(\mathrm{x}_1, \mathrm{x}_2, \mathrm{x}_3)$:

$$\mathrm{y}_1 = (\neg \mathrm{x}_1 \wedge \mathrm{x}_2) \vee (\mathrm{x}_1 \wedge \mathrm{x}_3)$$

$$\mathrm{y}_2 = (\neg \mathrm{x}_1 \wedge \mathrm{x}_2) \vee (\neg \mathrm{x}_2 \wedge \neg \mathrm{x}_3)$$

Aufgaben:

1. Wie viele Zeilen in der AND-Matrix (Produktterme) werden minimal benötigt?
2. Geben Sie für jeden Produktterm an, welche Eingangsleitungen (x_i bzw. $\neg \mathrm{x}_\mathrm{i}$) in der AND-Matrix verbunden werden.
3. Beschreiben Sie die Verschaltung der OR-Matrix zur Erzeugung von y_1 und y_2.

Musterlösung Szenario 2:

1. **Minimal benötigte Produktterme:** Der Term $(\neg \mathrm{x}_1 \wedge \mathrm{x}_2)$ tritt in beiden Funktionen identisch auf. Daher genügen **3 Produktterme**:

- $\mathrm{P}_1 = \neg \mathrm{x}_1 \wedge \mathrm{x}_2$ (geteilt von y_1 und y_2)
- $\mathrm{P}_2 = \mathrm{x}_1 \wedge \mathrm{x}_3$ (nur y_1)
- $\mathrm{P}_3 = \neg \mathrm{x}_2 \wedge \neg \mathrm{x}_3$ (nur y_2)

2. **Verbindungen in der AND-Matrix:**

- Zeile P_1: Kreuzungspunkte mit Spalte $\neg \mathrm{x}_1$ und Spalte x_2.

- Zeile P_2: Kreuzungspunkte mit Spalte x_1 und Spalte x_3.
- Zeile P_3: Kreuzungspunkte mit Spalte $\neg\mathrm{x}_2$ und Spalte $\neg\mathrm{x}_3$.

3. **Verschaltung der OR-Matrix:**
   - Ausgang y_1: Kreuzungspunkte mit Zeile P_1 und Zeile P_2 ($\mathrm{y}_1 = \mathrm{P}_1 \vee \mathrm{P}_2$).
   - Ausgang y_2: Kreuzungspunkte mit Zeile P_1 und Zeile P_3 ($\mathrm{y}_2 = \mathrm{P}_1 \vee \mathrm{P}_3$).

Simulator ↗

3.) Produktterm-Sharing zur Hardware-Minimierung

In programmierbaren Logikbausteinen (PLAs) ist die Anzahl der zur Verfügung stehenden Zeilen in der Konjunktionsmatrix (Produktterm-Budget) fest vorgegeben (z. B. maximal 4 oder 8 Produktterme).
Frage: Erklären Sie, warum bei der Minimierung für PLAs eine **gemeinsame Minimierung mehrerer Ausgänge** (Multi-Output Minimization) oft zu einem hardware-effizienteren Ergebnis führt als die getrennte KV-Minimierung jedes einzelnen Ausgangs.

Musterlösung Szenario 3:
Begründung für Multi-Output-Minimierung:

- **Begrenztes Produktterm-Budget:** Bei einem PLA bestimmt die Anzahl der AND-Gatter (Zeilen) die Chipfläche und Herstellbarkeit.
- **Suboptimalität isolierter KV-Diagramme:** Wird jeder Ausgang separat per KV-Diagramm minimiert, wählt man jeweils die maximal großen Blöcke (Primimplikanten). Diese Blöcke sind jedoch oft für verschiedene Ausgänge leicht versetzt, sodass keine Zeilen wiederverwendet werden können (z. B. 2 Terme für y_1 und 2 Terme für $\mathrm{y}_2 \implies$ 4 Zeilen).
- **Vorteil des Teilens (Sharing):** Wählt man bewusst einen etwas kleineren gemeinsamen Implikanten $\mathrm{P}_{\mathrm{comm}}$, der in beiden Funktionen enthalten ist, benötigt man: $\mathrm{y}_1 = \mathrm{P}_{\mathrm{comm}} \vee \mathrm{P}_{\mathrm{a}}$ und $\mathrm{y}_2 = \mathrm{P}_{\mathrm{comm}} \vee \mathrm{P}_{\mathrm{b}} \implies$ **nur 3 Zeilen** statt 4.
- **Fazit:** Die gemeinsame Ausnutzung identischer Zeilen reduziert die Zeilenanzahl der AND-Matrix und entscheidet oft darüber, ob eine Schaltung überhaupt in einen vorgegebenen Baustein passt.

Simulator ↗

4.) Don't-Cares in der Eingabe (BCD-Pseudotetraden)

Ein 4-Bit BCD-Signal ($\mathrm{x}_3\mathrm{x}_2\mathrm{x}_1\mathrm{x}_0$, mit x_3 als MSB) codiert Dezimalziffern von 0 bis 9 ($0000_2 \ldots 1001_2$).
Ein Fehlerdetektor ERR soll eine 1 ausgeben, wenn eine unzulässige Pseudotetrade ($10 \ldots 15$) anliegt ($1010_2 \ldots 1111_2$).
Aufgaben:

1. Formulieren Sie die minimierte DNF für ERR unter Ausnutzung von Don't Cares.
2. Geben Sie die entsprechende Spezifikation im Berkeley-Espresso-Format an. Wie viele Produktterme werden im PLA benötigt?

Musterlösung Szenario 4:

1. **Minimierte DNF für ERR:** Pseudotetraden sind: 10 (1010), 11 (1011), 12 (1100), 13 (1101), 14 (1110), 15 (1111).

- Belegungen 12..15 zeichnen sich durch $x_3 = 1 \wedge x_2 = 1$ aus $(11--)$.
- Belegungen 10..11 zeichnen sich durch $x_3 = 1 \wedge x_1 = 1$ aus $(1-1-)$.

$\Longrightarrow \mathrm{ERR} = (x_3 \wedge x_2) \vee (x_3 \wedge x_1) = x_3 \wedge (x_2 \vee x_1)$.

2. **Espresso-Notation (.pla):**

```
.i 4
.o 1
.ilb x3 x2 x1 x0
.ob ERR
.p 2
11-- 1
1-1- 1
.e
```

Es werden **nur 2 Produktterme** im PLA benötigt.

Simulator

5.) Systemvergleich: PROM vs. PAL vs. PLA

In der Digitaltechnik stehen verschiedene Architekturen programmierbarer Bausteine zur Verfügung.

Aufgaben:

1. Beschreiben Sie die Unterschiede bezüglich der Programmierbarkeit von AND- und OR-Matrix bei PROM, PAL und PLA.
2. Warum hat sich in der industriellen Praxis das PAL-Konzept historisch stärker verbreitet als das flexiblere PLA?

Musterlösung Szenario 5:

1. **Architekturvergleich:**

- **PROM:** Feste AND-Matrix (2^n-Decodierer für alle Minterme), programmierbare OR-Matrix. Vorteil: Einfach als Look-Up-Table nutzbar. Nachteil: Exponentieller Zeilenaufwand bei vielen Eingängen (2^n).
- **PAL (Programmable Array Logic):** Programmierbare AND-Matrix, fest verdrahtete OR-Matrix. Vorteil: Schnell, kostengünstig zu fertigen. Nachteil: Keine Wiederverwendung von Produktermen zwischen verschiedenen Ausgängen.
- **PLA:** Sowohl AND- als auch OR-Matrix programmierbar. Vorteil: Maximale Flexibilität, Produktterm-Sharing. Nachteil: Etwas langsamere Signallaufzeit durch zwei programmierbare Kreuzungsebenen.

2. **Verbreitung des PAL:**

- **Herstellungskosten & Ausbeute:** Da nur eine Matrix (AND) programmierbare Fuses benötigte, war der Fertigungsprozess robuster und günstiger.
- **Geschwindigkeit (Timing):** Feste OR-Gatter weisen geringere parasitäre Kapazitäten auf und schalten schneller als konfigurierbare Matrizen.